Glossary

Abbreviations for Peripherals

2D-DMA	2D-DMA Controller
ADC	ADC Controller
AES	AES (Advanced Encryption Standard) Accelerator
DMA	DMA (Direct Memory Access) Controller
ECC	ECC (Elliptic Curve Cryptography) Accelerator
ECDSA_DS	ECDSA Digital Signature Peripheral
eFuse	eFuse Controller
EMAC	Ethernet Media Access Controller
ETM	Event Task Matrix
HMAC	HMAC (Hash-based Message Authentication Code) Accelerator
I2C	I2C (Inter-Integrated Circuit) Controller
I2S	I2S (Inter-IC Sound) Controller
I3C	I3C (Improved Inter-Integrated Circuit) Controller
ISP	Image Signal Processor
LCD_CAM	LCD and Camera Controller
LEDC	LED PWM (Pulse Width Modulation) Controller
MCPWM	Motor Control PWM (Pulse Width Modulation)
PARLIO	Parallel IO Controller
PCNT	Pulse Count Controller
PIE	Processor Instruction Extensions
PMS	Permission Control
PPA	Pixel-Processing Accelerator
RMT	Remote Control Peripheral
RNG	Random Number Generator
RSA	RSA (Rivest Shamir Adleman) Accelerator
RSA_DS	RSA Digital Signature Peripheral
SDHOST	SD/MMC Host Controller
SHA	SHA (Secure Hash Algorithm) Accelerator
SPI	SPI (Serial Peripheral Interface) Controller
TIMG	Timer Group
TOUCH	Touch Sensor
TRACE	RISC-V Trace Encoder
TSENS	Temperature Sensor
TWAI	Two-Wire Automotive Interface
UART	UART (Universal Asynchronous Receiver-Transmitter) Controller
ULP Coprocessor	Ultra-low-power Coprocessor
USB OTG	USB On-The-Go
USB_SERIAL_JTAG	USB Serial/JTAG Controller
VAD	Voice Activity Detection
VDMA	VDMA Controller
WDT	Watchdog Timers

XTS_AES External Memory Encryption and Decryption

Abbreviations Related to Registers

REG **Register.**

SYSREG **System registers** are a group of registers that control system reset, memory, clocks, software interrupts, power management, clock gating, etc.

ISO **Isolation.** If a peripheral or other chip component is powered down, the pins, if any, to which its output signals are routed will go into a floating state. ISO registers isolate such pins and keep them at a certain determined value, so that the other non-powered-down peripherals/devices attached to these pins are not affected.

NMI **Non-maskable interrupt** is a hardware interrupt that cannot be disabled or ignored by the CPU instructions. Such interrupts exist to signal the occurrence of a critical error.

W1TS Abbreviation added to names of registers/fields to indicate that such register/field should be used to set a field in a corresponding register with a similar name. For example, the register `GPIO_ENABLE_W1TS_REG` should be used to set the corresponding fields in the register `GPIO_ENABLE_REG`.

W1TC Same as *W1TS*, but used to clear a field in a corresponding register.

Access Types for Registers

Sections *Register Summary* and *Register Description* in TRM chapters specify access types for registers and their fields.

Most frequently used access types and their combinations are as follows:

- RO
- WO
- WT
- R/W
- R/W1
- WL
- R/W/SC
- R/W/SS
- R/W/SS/SC
- R/WC/SS
- R/WC/SC
- R/WC/SS/SC
- R/WS/SC
- R/WS/SS
- R/WS/SS/SC
- R/SS/WTC
- R/SC/WTC
- R/SS/SC/WTC
- RF/WF
- R/SS/RC
- varies

Descriptions of all access types are provided below.

R **Read.** User application can read from this register/field; usually combined with other access types.

RO **Read only.** User application can only read from this register/field.

HRO **Hardware Read Only.** Only hardware can read from this register/field; used for storing default settings for variable parameters.

W **Write.** User application can write to this register/field; usually combined with other access types.

WO **Write only.** User application can only write to this register/field.

W1 **Write Once.** User application can write to this register/field only once; only allowed to write 1; writing 0 is invalid.

SS **Self set.** On a specified event, hardware automatically writes 1 to this register/field; used with 1-bit fields.

SC **Self clear.** On a specified event, hardware automatically writes 0 to this register/field; used with 1-bit and multi-bit fields.

SM **Self modify.** On a specified event, hardware automatically writes a specified value to this register/field; used with multi-bit fields.

SU **Self update.** On a specified event, hardware automatically updates this register/field; used with multi-bit fields.

RS **Read to set.** If user application reads from this register/field, hardware automatically writes 1 to it.

RC **Read to clear.** If user application reads from this register/field, hardware automatically writes 0 to it.

RF **Read from FIFO.** If user application writes new data to FIFO, the register/field automatically reads it.

WF **Write to FIFO.** If user application writes new data to this register/field, it automatically passes the data to FIFO via APB bus.

WS **Write any value to set.** If user application writes to this register/field, hardware automatically sets this register/field.

W1S	**Write 1 to set.** If user application writes 1 to this register/field, hardware automatically sets this register/field.
W0S	**Write 0 to set.** If user application writes 0 to this register/field, hardware automatically sets this register/field.
WC	**Write any value to clear.** If user application writes to this register/field, hardware automatically clears this register/field.
W1C	**Write 1 to clear.** If user application writes 1 to this register/field, hardware automatically clears this register/field.
W0C	**Write 0 to clear.** If user application writes 0 to this register/field, hardware automatically clears this register/field.
WT	**Write 1 to trigger an event.** If user application writes 1 to this field, this action triggers an event (pulse in the APB bus) or clears a corresponding WTC field (see WTC).
WTC	**Write to clear.** Hardware automatically clears this field if user application writes 1 to the corresponding WT field (see WT).
W1T	**Write 1 to toggle.** If user application writes 1 to this field, hardware automatically inverts the corresponding field; otherwise - no effect.
W0T	**Write 0 to toggle.** If user application writes 0 to this field, hardware automatically inverts the corresponding field; otherwise - no effect.
WL	**Write if a lock is deactivated.** If the lock is deactivated, user application can write to this register/field.
varies	**The access type varies.** Different fields of this register might have different access types.